

HIGH PERFORMANCE COMPUTING

Assignment - 6

Name : Thota Sai Teja

HT NO : 2303A51535

Batch No : 22

NUMA Effects and Bandwidth Measurement Objective: To observe how memory placement and NUMA effects influence performance and to understand bandwidth limitations.

Task 1: System NUMA Information lscpu | grep -i numa Observation: Number of NUMA nodes = ____ **Code :**

```
%%bash
# Install NUMA development libraries if not already installed
apt-get update -qq > /dev/null apt-get install -y libnuma-
dev > /dev/null

# Write the C code to a file
cat << EOF > numa_info.c
#include <stdio.h>

#include <numa.h>

int main() {

    if (numa_available() < 0) { printf("NUMA is not
        available on this system.\n"); return 1;
    }

    int nodes = numa_max_node() + 1; int
    cpus = numa_num_configured_cpus();

    printf("NUMA is available on this system.\n");
    printf("Number of NUMA nodes = %d\n", nodes);
    printf("Number of CPUs = %d\n", cpus);

    for (int i = 0; i < nodes; i++) {
        printf("Node %d has %lld MB memory\n",
            i,
            numa_node_size(i, NULL) / (1024 * 1024));
    }

    return 0;
}
EOF
# Compile the C code gcc
numa_info.c -o numa_info -lnuma
```

Execute the compiled program

./numa_info **OUTPUT:**

NUMA is available on this system.

Number of NUMA nodes = 1

Number of CPUs = 2

Node 0 has 12975 MB memory

W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide it (sources.list entry misspelt?)

Task 2: First Touch Memory Initialization File: numa_first_touch.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <omp.h>
```

```
#define N 10000000 06-02-2026 EOD int main() { double A
```

```
= (double)malloc(sizeof(double)*N); #pragma omp parallel
```

```
for for (int i = 0; i < N; i++) A[i] = 1.0; double sum = 0;
```

```
#pragma omp parallel for reduction(+:sum) for (int i = 0; i < N; i++) sum += A[i];
```

```
printf("Sum = %f\n", sum); free(A); return 0; } Compile & Run: gcc -O2 -fopenmp
```

```
numa_first_touch.c -o numa_first_touch ./numa_first_touch Code : %%bash
```

```
cat << EOF > numa_first_touch.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <omp.h>
```

```
#include <time.h>
```

```
#define N 10000000
```

```
int main() { double
```

```
*A =
```

```
(double*)malloc(sizeo
```

```
f(double) * N);
```

```
if (A == NULL) {
```

```

        printf("Memory allocation failed\n");
return 1; } double start, end;

// First Touch Initialization
start = omp_get_wtime();

#pragma omp parallel for
for (int i = 0; i < N; i++) {
    A[i] = 1.0;
}

end = omp_get_wtime();
printf("Initialization Time = %f seconds\n", end - start);

// Summation double sum
= 0.0; start =
omp_get_wtime();

#pragma omp parallel for
reduction(+:sum) for (int i = 0; i < N;
i++) { sum += A[i];
} end =

omp_get_wtime();

printf("Sum = %f\n", sum);
printf("Summation Time = %f seconds\n", end - start);

free(A);
return 0;
}

```

OUTPUT :

EOF

```
gcc -O2 -fopenmp numa_first_touch.c -o numa_first_touch -lm
./numa_first_touch
```

Initialization Time = 0.029790 seconds

Sum = 10000000.000000

Summation Time = 0.008760 seconds

./numa_first_touch Task 3:

Conceptual Questions

1. What is first-touch policy?

1. Does NUMA affect correctness or performance?
2. Why is memory bandwidth important in NUMA systems?

%%bash

Code :

```
cat << EOF > memory_bandwidth.c
#include <stdio.h>
#include <stdlib.h>

#include <omp.h>

#define N 50000000

int main() {

    double *A = (double*)malloc(sizeof(double) * N);
    double *B = (double*)malloc(sizeof(double) * N);

    if (A == NULL || B == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    // Initialize arrays
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        A[i] = 1.0;
        B[i] = 2.0; } double start
= omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        A[i] = A[i] + B[i];
    } double end =

    omp_get_wtime(); double time

= end - start;

    double bytes_accessed = 3 * sizeof(double) * N;
    double bandwidth = (bytes_accessed / time) / 1e9;
    printf("Execution Time = %f seconds\n", time);
    printf("Estimated Memory Bandwidth = %f GB/s\n",
    bandwidth);

    free(A);
    free(B);
```

```
    return 0;  
}
```

OUTPUT:

EOF

```
gcc -O2 -fopenmp memory_bandwidth.c -o memory_bandwidth -lm  
./memory_bandwidth
```

Execution Time = 0.091972 seconds

Estimated Memory Bandwidth = 13.047410 GB/s